

- Edge cases:

- data kosong
- concurrency
- Manual testing (user flow)
- API test (Postman / automated)
- Unit test (core logic)

- Domain + SSL

- Reverse proxy:
 - Nginx
- Deployment:
 - VPS / cloud (AWS, GCP, dll)
- Environment:
 - dev / staging / production
- CI/CD:
 - auto deploy (optional tapi bagus)
- Version control:
 - Git + branching strategy

- Alert system (optional)

- Logging:
 - request log
 - error log
- Server monitoring:
 - CPU, memory
- Error tracking:
 - Sentry (recommended)

Supaya tahu kalau ada error.

- Data seeding

- Migration strategy
- Restore testing
- Backup otomatis (harian)

- Feature documentation

- Database schema doc
- Setup guide
- API documentation

Perlu agar scalable dan mudah dilaniutkan/kembangkan.

— — — — —

Akses Sangat Terbatas: Hanya bisa membuat pesanan baru (POS), memindahkan kartu di Kanban board, memproses pembayaran, dan menutup shift mereka sendiri. Mereka **tidak bisa** melihat sisa stok secara detail (hanya peringatan jika habis) dan **tidak bisa** membatalkan transaksi yang sudah dibayar tanpa PIN dari Manager.

- Platform Sistem **Multi-tenant**, (=adalah arsitektur perangkat lunak di mana banyak pelanggan (atau "**tenant**") berbagi **aplikasi** dan infrastruktur yang sama secara aman, sambil tetap menjaga pemisahan data, konfigurasi, dan akses pengguna mereka.)
- Komunikasi aplikasi dengan data:
 - REST API** (Untuk versi 1.0) [yang menggunakan HTTP standar (GET, POST, PUT, DELETE)]
 - Gunakan **GraphQL** kalau aplikasi sudah besar dan melayani 1000+ pengguna
- Stack:
 - Frontend: **React + Vite**
 - Deployment Front-End: **Vercel** atau **Netlify**
 - Backend: **Node.js + Express**
 - Deployment Back-End: **Render.com** atau **Railway.app**
 - Database: **Supabase**
 - Penyimpanan File (Gambar/produk bukti bayar): **Cloudinary**
- Arsitektur Aplikasi:
 - Monolith**

4. Security Design (Database Level)

Sistem ERP adalah aplikasi padat data (*data-heavy system*). Desain struktur basis data (DB) harus dikunci dengan sangat matang sejak hari pertama untuk mencegah dua bencana utama: kebocoran data antar toko (UMKM) dan kecurangan operasional (*fraud*).

1. Keamanan SaaS Multi-Tenant (Perisai Utama)

- Tenant Isolation (Isolasi Penyewa):** Wajib menambahkan kolom `tenant_id` di setiap tabel operasional (seperti *Orders*, *Products*, *Inventory_Logs*). Ini adalah fondasi mutlak agar data Toko A tidak pernah tercampur dengan data Toko B.
- Row Level Security (RLS) Supabase:** Keamanan diimplementasikan secara langsung di jantung *database*, bukan hanya di API.
 - Penerapan:** Buat kebijakan (*policy*) yang menyatakan: "*Izinkan operasi SELECT/INSERT/UPDATE/DELETE HANYA JIKA `tenant_id` pada baris data tersebut cocok dengan sesi pengguna yang sedang login*".
 - Benefit:** Jika API *Backend* berhasil dibobol peretas, data tetap aman karena *database* secara otomatis menolak akses di level baris data.
- Idempotency Keys (Anti-Transaksi Ganda):** Desain sistem untuk menangani gangguan koneksi. *Backend* harus mengenali ID permintaan yang sama (misal: ID Pesan dari WhatsApp atau ID Transaksi Kasir). Jika tombol "Bayar" ditekan dua kali berturut-turut akibat *lag*, sistem **tidak akan** memotong stok atau mencatat pendapatan dua kali.

2. Integritas Data & Audit Trail (Anti-Kecurangan)

- Soft Delete vs Hard Delete:** Haram melakukan *hard delete* (perintah `DELETE` permanen ke *database*). Jika sebuah produk atau menu dihapus, riwayat transaksi bulan lalu akan rusak.
 - Solusi:** Selalu gunakan *soft delete*. Data hanya disembunyikan dari antarmuka aplikasi kasir (dengan mengisi tanggal pada `deleted_at`), namun fisik datanya tetap utuh di *database* untuk keperluan audit laporan keuangan.
- Audit Fields (Ekspansi Penuh):** Mengetahui *kanan* data berubah tidaklah cukup. Anda harus tahu *siapa aktornya* untuk mencegah penggelapan. Setiap tabel operasional wajib memiliki:
 - `created_at` & `created_by` (*Siapa kasir yang menginput*)
 - `updated_at` & `updated_by` (*Siapa manajer yang mengedit*)
 - `deleted_at` & `deleted_by` (*Siapa yang membatalkan/void transaksi*)

3. Optimasi Arsitektur & Performa

- Indexing:** Wajib diterapkan pada kolom yang sering dijadikan parameter pencarian atau filter (terutama `tenant_id`, `created_at`, dan kolom *Foreign Key*). Ini akan memangkas waktu pencarian jutaan baris data dari hitungan detik menjadi milidetik.
- Normalisasi (Minimal 3NF):** *Third Normal Form* (3NF) memastikan tidak ada redundansi data. (Contoh: Jangan menyimpan teks "Kategori Kopi Susu" berulang-ulang di setiap baris tabel produk. Simpan ID-nya saja, dan relasikan ke tabel khusus `Categories`).
- Dokumentasi Peta Data:** ERD (Entity Relationship Diagram) wajib dipetakan dan dikunci sebelum mengeksekusi migrasi tabel pertama kali di Supabase.

4. Relasi Penting (Core Relations)

Berikut adalah relasi entitas inti yang menyatukan aliran data Micro-ERP Anda:

- Tenants → Users:** (Dihubungkan melalui tabel pivot `tenant_users` untuk mengendalikan *Role-Based Access Control* / RBAC tingkat toko).
- Orders → Order_Items:** (Pesanan ke Detail Pesanan. Berfungsi untuk mengunci harga

- Refactoring rutin
- Comment secukupnya
- Modular structure
- Clean code (naming konsisten)

Note:

Supaya project tidak "mati" setelah launch.

Urutan build terbaik:

- Auth + RBAC
- Master data (user, customer, product)
- Transaction core (invoice / inventory)
- Reporting sederhana
- Baru scaling fitur lain

Urutan build terbaik:

Saran Strategy (biar realistis)

ERP yang setengah jadi = tidak akan dipakai user.

- Fokus:
 - correctness > banyak fitur
- Mulai dari:
 - 1 module solid (misalnya invoicing + inventory)
- Jangan langsung bikin "ERP lengkap"

Sebagai solo dev:

Real Talk (Penting)

- Idempotency (hindari double submit)
- Data locking (untuk finance)
- Audit log:
 - siapa edit apa
- Transaction consistency
 - gunakan DB transaction

Checklist:

Ini pembeda ERP vs web biasa.

barang pada saat transaksi terjadi, sehingga perubahan harga di masa depan tidak merusak laporan historis).

3. **Products → Inventory_Logs:** (Katalog barang dihubungkan ke riwayat rekam jejak. Stok tidak hanya berupa angka yang di-*update*, melainkan aliran mutasi keluar/masuk yang tercatat secara atomik).

5. Backend Architecture (Node.js + Express)

✗ Hal yang Tidak Dikerjakan di Backend

1. Manual Password Hashing & Salting

- Status: ❌ Dihapus / Dialihkan.
- Alasan: Tidak perlu instal `bcrypt` atau menulis logika enkripsi sendiri. Supabase Auth sudah menangani enkripsi kata sandi (algoritma tingkat tinggi seperti `Argon2/bcrypt`) secara otomatis di latar belakang.

2. Session-Based Authentication (Cookies)

- Status: ❌ Tidak digunakan.
- Alasan: Karena sistem kita menggunakan *decoupled architecture* (API terpisah), kita menggunakan **Stateless JWT** (JSON Web Token) yang di-*generate* langsung oleh Supabase.

3. Endpoint Registrasi Publik (Public Sign-Up)

- Status: ❌ Dihapus / Tidak Digunakan.
- Alasan: Sistem menerapkan "Sales-Led Onboarding". Backend menolak pendaftaran akun liar dari luar. Akun *Tenant* dan *Owner* hanya di-*generate* dari sisi internal (Supabase Admin API), guna menjaga kebersihan *database* dari *spam* dan akun coba-coba.

✅ Backend Core Architecture

1. Identity & Access (RBAC)

- Stack: Supabase JWT & Express Custom Middleware.
- Rules:
 - Setiap *request* ke *endpoint* privat **wajib** menyertakan Header: `Authorization: Bearer <token>`.
 - *Middleware* wajib memvalidasi token **DAN** peran (*role*) pengguna.

Flow Authorization:

```
Request Masuk
↓
Cek JWT Header
↓
Valid? —(No)—> 401 Unauthorized
↓ (Yes)
Cek tabel tenant_users (Role & Tenant ID)
↓
Role Sesuai? —(No)—> 403 Forbidden
↓ (Yes)
Lanjut ke Controller (200 OK)
```

2. Infrastructure Protection

A. Environment Variables (`.env`)

- Rules: Segala jenis *secret key* (seperti `SUPABASE_SERVICE_KEY`, *Webhook Token* WhatsApp) **wajib** diletakkan di `.env`.
- 🚫 **Strict Warning:** File `.env` **HARAM** di-*commit* ke GitHub. Selalu gunakan `.gitignore`.

B. CORS (Cross-Origin Resource Sharing)

- Rules: Backend Express harus menolak *request* dari domain yang tidak dikenal.
- Setup: Masukkan ke *whitelist* hanya URL domain *frontend* produksi (misal: `<https://kasir-umkm.com>`) dan `localhost` untuk fase *development*.

C. Rate Limiting

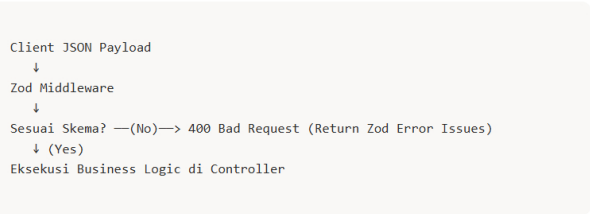
- Rules: Wajib dipasang pada *endpoint* publik, terutama URL Webhook WhatsApp.
- Goal: Mencegah serangan DDoS atau *spam* pesan beruntun yang berpotensi menyebabkan OOM (*Out of Memory*).

3. Data Flow & Validation

- Stack: Zod
- Rules: Tidak boleh ada `req.body` yang masuk ke *Controller* tanpa melewati validasi Skema Zod. Cegah masuknya data cacat (misal: `total_price` diisi *string*, padahal DB

meminta *integer*).

Flow Validation:



4. Routing & Error Handling

A. API Versioning

- Rules: Semua rute API wajib menggunakan konvensi *prefix* versi (Contoh: `/api/v1/orders` , `/api/v1/inventory`).
- Alasan: Memudahkan pembuatan versi API baru di masa depan (misal untuk *mobile app*) tanpa merusak ekosistem *web* kasir yang sudah berjalan di `v1` .

B. Centralized Error Handling

- Rules: Buat satu *file global error handler middleware* di Express.
-  **Strict Warning:** JANGAN PERNAH mengembalikan *stack trace* (jalur *error* spesifik DB) ke *response* JSON Client.
- **Response Standar:** Jika *database* gagal, catat detailnya secara diam-diam di *server log*, lalu kembalikan *response* abstrak ke *frontend*: `500 Internal Server Error` .

 Core Backend Philosophy

Prinsip	Definisi & Aturan
Thin Backend	Beban otentikasi diserahkan ke Supabase, <i>backend</i> fokus pada validasi & logika bisnis UMKM.
Fail Fast	Tolak <i>request</i> secepat mungkin di level <i>middleware</i> jika token salah atau <i>payload</i> cacat.
Zero Trust	Jangan pernah percaya data yang dikirim dari <i>Frontend</i> . Selalu validasi ulang di <i>Backend</i> .
Tenant Isolation	Setiap <i>query</i> DB WAJIB menyertakan <code>tenant_id</code> untuk mencegah kebocoran data antar toko.
Silent Failures	Error sistem internal hanya untuk mata <i>developer</i> (via log), jangan bocor ke <i>client</i> .

6. Frontend Development Guidelines

 Hal yang Tidak Digunakan di Frontend

1. API Versioning (`/v1`)

- Status:  Tidak di-*handle* di Frontend.
- Alasan: *Versioning* murni tanggung jawab Backend/API Server. Frontend hanya bertugas memanggil *endpoint* dan mengonsumsi *response*.

2. Controller & Repository Pattern

- Status:  Tidak digunakan.
- Alasan: React dibangun di atas paradigma **Components, Hooks**, dan **UI Composition**. Menerapkan pola kelas *Controller/Repository* hanya akan menambah *boilerplate*, kompleksitas, dan *duplicated logic*.

3. Heavy Business Logic

- Status:  Dihindari.
- Alasan: *Business rules* utama harus dipusatkan di Backend. Frontend dibatasi hanya untuk:
 - *Rendering UI*
 - *Client validation*
 - *Fetch state*
 - *Interaction state*

4. Halaman Registrasi SaaS & Master Admin Panel

- Status:  Tidak Dibuat untuk Fase MVP.
- Alasan: Memangkas waktu *development* (Scope-cutting). Seluruh sumber daya UI *Frontend* 100% difokuskan untuk layar operasional UMKM (Kanban, POS, Dashboard). Manajemen toko tingkat platform oleh Superadmin cukup memanfaatkan UI bawaan dari Supabase Studio.

✔ Frontend Core Architecture

1. Validation Layer

- Stack: `React Hook Form` + `Zod`
- Rules:
 - Semua *form* wajib menggunakan *schema validation*.
 - Tidak boleh ada *direct API submit* tanpa validasi lokal.
- Use Cases: Tambah produk, Checkout POS, Edit Inventory, User Management.

Flow Validation: `User Input` → `React Hook Form` → `Zod Validation` → `API Request`

2. Data Fetching & Caching

- Stack: `TanStack Query`
- Rules: Hindari pemanggilan data (*fetch*) manual menggunakan `useEffect`. Semua *server state* harus menggunakan Query Hooks.
- Benefit:
 - `TanStack Query` menangani *caching*, *loading/error state*, dan *background refetch*.
 - Navigasi antar halaman terasa instan dan mengurangi beban *request database*.

3. API Layer

- Structure: Terpusat pada folder `services/` (contoh: `auth.service.ts`, `product.service.ts`).
- Rules: Folder ini hanya boleh berisi fungsi *fetch* atau pemanggilan Supabase API. Haram mencampurkan *UI logic*, *component state*, atau *business rules* kompleks di dalamnya.

4. Global Error Handling

- Layer 1 (Error Boundary): *Root app* wajib dibungkus `ErrorBoundary` untuk mencegah aplikasi berubah menjadi layar putih (*blank*) jika terjadi *runtime error*.
- Layer 2 (Toast Notifications): Gunakan `Sonner` atau `Shadcn Toast` untuk *feedback non-blocking* (misal: gagal simpan data, internet putus). Haram menggunakan `alert()` bawaan *browser* atau menampilkan *raw error text*.

5. Pagination & URL State

- Rules: Segala bentuk filter wajib disimpan di parameter URL (contoh: `?page=2&kategori=kopi`).
- Alasan: Menjamin agar *state* tidak hilang saat pengguna melakukan *refresh*, URL dapat dibagikan (*shareable*), dan navigasi panah *back/forward* browser tetap sinkron.

6. Route Protection

- Stack: `<ProtectedRoute />` component + `Zustand` auth state.
- Rules: Pengguna tanpa sesi/token dilarang keras mengakses rute privat (`/dashboard`, `/pos`, dll).

Flow Security: `User Access Route` → `ProtectedRoute Check` → `Has Token?` → (Yes) `Render Page` / (No) `Redirect /login`

7. Code Splitting & Lazy Loading

- Stack: `React.lazy()` + `<Suspense />`
- Rules: Setiap modul raksasa (Kasir, Inventaris, Laporan) wajib di-*lazy-load*.
- Benefit: *Initial load* web terasa jauh lebih cepat, ukuran *bundle* kecil, dan sangat bersahabat untuk koneksi internet lambat.

8. Progressive Web App (PWA)

- Stack: `vite-plugin-pwa` (Membutuhkan `manifest.json` & `service-worker.js`).
- Goal: Web ERP ini dirancang agar dapat di-*install* langsung ke layar utama *smartphone* staf UMKM selayaknya aplikasi *native*, lengkap dengan fitur *offline caching*, tanpa perlu menyewa *developer* Android/iOS terpisah.

9. Real-Time Synchronization

- Stack: `Supabase Realtime` (WebSockets)
- Rules: Gunakan *subscription channel* HANYA untuk modul yang sifatnya kolaboratif/live (misal: Kanban Board pesanan WhatsApp). Hindari *me-render* ulang seluruh layar; cukup perbarui *state Zustand* atau batalkan *cache (invalidate)* di `TanStack Query`.

🧠 Core Frontend Philosophy

Elemen Sistem	Standar yang Digunakan (<i>Rules</i>)
Server State	TanStack Query
Global State	Zustand
Form State	React Hook Form
Validation	Zod

UI Components	Shadcn/ui
Styling	Tailwind CSS
Routing	React Router
Notifications	Toast (Sonner)
Loading State	Skeleton (Hindari <i>Spinner-only</i>)
Large Modules	Lazy Loading (React.lazy)

7. Integration Layer

1. Design System

- Stack: `Tailwind CSS` + `Shadcn/ui`
- Rules:
 - Hindari membuat CSS *custom* dari nol kecuali dalam kondisi kritis.
 - Gunakan *CSS Variables* melalui `tailwind.config.js` (Cukup tentukan 1 **Primary Color** & 1 **Accent Color**).
 - Light/Dark mode* sepenuhnya mengandalkan sistem *Shadcn* secara otomatis.
- Tipografi (Font): Gunakan font standar dengan keterbacaan tinggi seperti **Inter** atau **Geist**.

2. State Management & Form Handling

- Global State (Zustand)**: Digunakan murni untuk *Session Login*, *POS Cart State*, dan *Temporary UI State*.
- Form Handling (RHF + Zod)**: Segala bentuk *input* pengguna harus melewati jalur validasi yang ketat sebelum menyentuh lapisan *Backend*.

3. Responsive Strategy (Context-First Layout)

ERP tidak menggunakan pendekatan *Mobile-First* murni. Tata letak harus sadar akan konteks penggunaanya:

- Dashboard & Reports**: Dioptimalkan untuk **Mobile**. Pemilik (Owner) biasanya mengakses metrik omzet dan ringkasan kas masuk secara cepat lewat *smartphone*.
- Kanban / WA Orders**: Dioptimalkan untuk **Desktop / Wide Screen**. Menggeser (*drag-and-drop*) puluhan kartu pesanan di layar kecil sangat rentan *error*.
- POS Module (Kasir)**: Dioptimalkan untuk **Tablet Landscape**. Kasir menuntut area yang luas agar secara bersamaan bisa memantau *Product List*, *Cart*, dan *Quick Actions* tanpa perlu melakukan *scroll*.

4. Loading & Error States

- Loading State**: Tinggalkan layar putih. Gunakan komponen `<Skeleton />` agar antarmuka seolah sudah memuat strukturnya, memberikan ilusi aplikasi yang berjalan secepat kilat.
- Error State**: Rancanglah *friendly empty state* dengan tombol *Retry*. Jangan pernah menakut-nakuti kasir UMKM dengan teks *error debug* teknis berwarna merah atau layar macet total.

5. Dashboard UX (High-Density, Low-Clutter)

Sistem ERP membutuhkan visualisasi ribuan titik data secara bersamaan.

- Data Density**: Gunakan ukuran teks efisien (`14px` untuk tabel) dengan *row height* yang cukup rapat. Kurangi area kosong (*whitespace*) yang tidak perlu agar UMKM tidak letih melakukan *scroll*.
- Call to Action (CTA) Priority**:
 - Dalam satu layar pengerjaan, **HANYA** boleh ada **1 tombol aksi utama (Primary)** yang mencolok (Contoh: tombol solid `Buat Pesanan`).
 - Sisanya harus direduksi menjadi aksi sekunder (*outline, gray, ghost button*).

🧠 Core Integration Principles

Prinsip UX/UI	Target Ekspektasi
Consistency	Gunakan pola visual dan interaksi yang identik di seluruh modul ERP.
Speed	Minimalikan penulisan kode <i>styling</i> & <i>logic</i> antarmuka secara manual.
Readability	UI harus sanggup menampung kepadatan data tinggi namun tetap terasa bersih.
Validation First	"Saring di pintu depan": Semua kotak <i>input</i> wajib tervalidasi seketika.
Context-Aware	Desain menyesuaikan <i>device</i> : Kasir butuh layar lebar (Tablet), Bos butuh cepat (Mobile).

8. WhatsApp Integration Architecture (Core Feature)

Sistem pesanan via WhatsApp (Omnichannel Kanban) adalah fitur unggulan Micro-ERP ini. Mengingat status proyek masih di tahap MVP dan *Zero-Budget*, kita menggunakan pendekatan alternatif yang tangguh namun tetap mempersiapkan jalur migrasi ke tingkat *Enterprise*.

🔧 Strategi Integrasi (The Approach)

1. Unofficial Web Protocol (Fase MVP)

- **Stack:** `Baileys` atau `whatsapp-web.js` (Berjalan di atas Node.js).
- **Status:** ● Digunakan.
- **Alasan:** 100% Gratis, tidak memerlukan verifikasi dokumen bisnis Meta (Facebook), dan memungkinkan tim untuk memvalidasi fitur Kanban secara instan. Sistem bekerja dengan cara men- *generate* QR Code di peladen (*server*), yang kemudian dipindai oleh *smartphone* admin UMKM.

2. The "Adapter Pattern" (Persiapan Migrasi)

- **Rules:** Haram memanggil fungsi pustaka WhatsApp secara langsung di dalam *Controller* Express.
- **Alasan:** Suatu saat UMKM akan membesar dan membutuhkan **Meta Cloud API** resmi. Dengan menggunakan *Adapter Pattern* (membungkus logika WA ke dalam satu *interface/service* khusus), migrasi ke API resmi nanti tidak akan merusak logika bisnis pesanan (Order) atau *database* Anda.